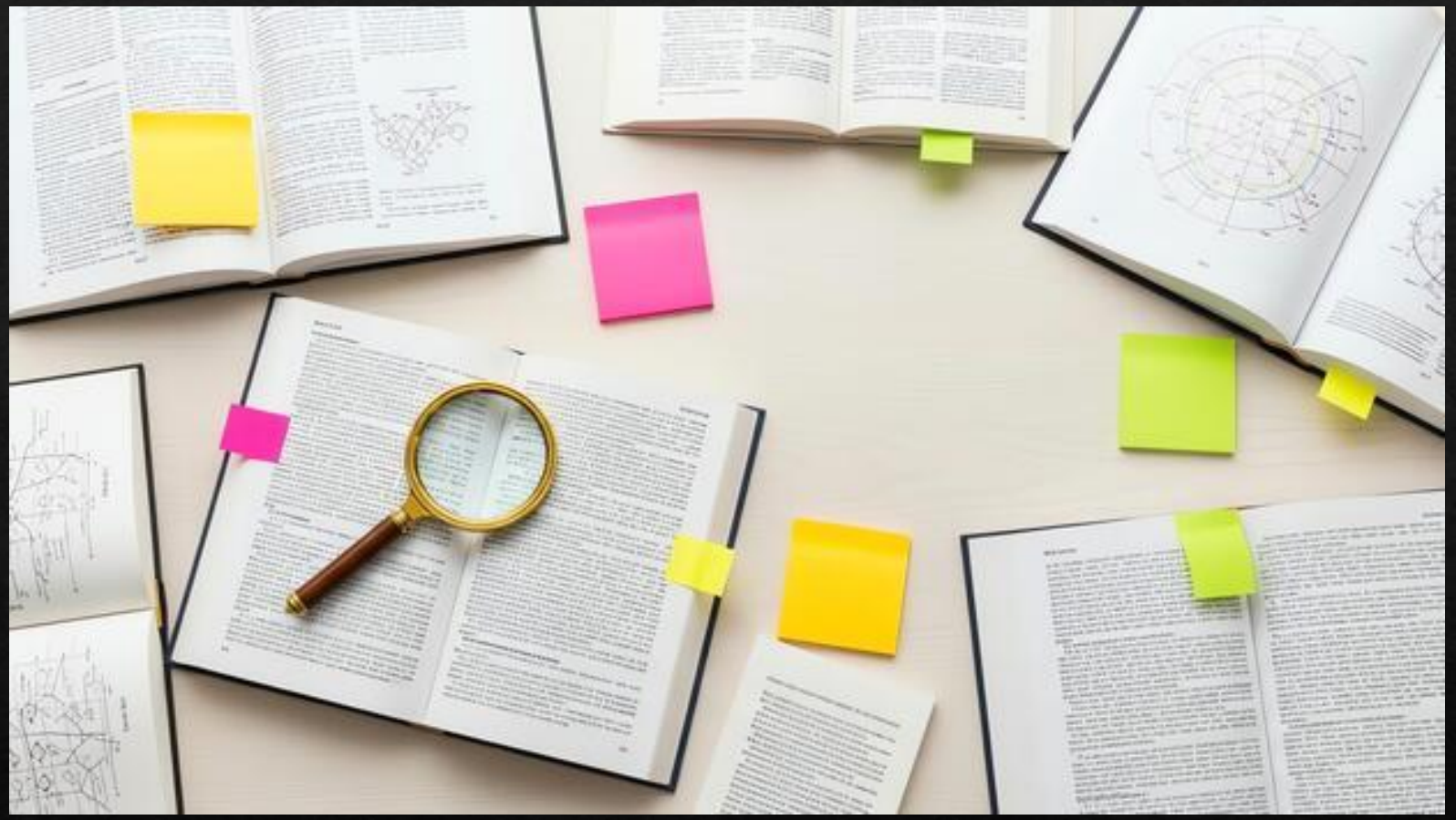
Indexing



Search a topic without index

Search a topic with index

CONTENTS

How to trade stock indices 04
What are stock indices. 04
Why trade stock indices 05
How is the value of stock indices determined? 06
Calculation based on the market capitalization of companies 06
Calculation based on the prices of individual shares 06
What affects the price movement of stock indices. 07
An overview of the most prominent indices and their trading hours. 07
Cross index correlation. 10
Stock indices and bond interest rates. 11
Technical specifications for stock index trading 12
Leverage. 12
Margin 12
Fees, spreads, and swaps 13
Stock index trading strategy 15
Swing strategy 15
Perception of the long-term market trend 15
Monitoring of 100 daily and 200 daily moving average 16
Use on European indices 17
Confidence in long-term growth or confirmation? 17
Trading position management 18
Intraday Price Action strategy 19
Pullback 19
Confirmation of divergence 20
Enter based on pullback and divergence. 21
Bonus and other examples 23
Position management 25

Imagine karo apke paas 10 lakh users ka data hai

aur aap query chalate ho:

```
db.users.find({ email: "abc@gmail.com" })
```

Agar indexing nahi hai… to MongoDB pura database scan karega

But agar indexing hai… to direct result milta hai in milliseconds

What is Indexing?

Simple language me Index ek shortcut / reference hota hai Jaise book me index hota hai (page number find karne ke liye)

Without Index: Full Collection Scan (slow ❌)

With Index: Direct search (fast ☑)

How Index Works Internally

MongoDB uses B-Tree structure

Data sorted form me store hota hai

Example: db.users.createIndex({ email: 1 })

1 = ascending

-1 = descending

Types of Indexes (Core Part)

1. Single Field Index
2. Compound Index
3. Multikey Index
4. Text Index

1. Single Field Index

Ek field pe index

```
db.users.createIndex({ email: 1 })
```

Use case:
Search by one field

2. Compound Index

Multiple fields pe index

```
db.users.createIndex({ name: 1, age: -1 })
```

Important:

- ❑ Order matters

Works for:

- ❑ Name
- ❑ name + age
- ❑ ❌ Not for only age

3. Multikey Index

Created on an array field

```
{ name: "Manas", skills: ["JS", "MongoDB"] }
```

Example data:

```
db.students.createIndex({ skills: 1 })
```

MongoDB automatically multiple entries banata hai

4. Text Index

Full-text search ke liye

```
db.posts.createIndex({ content: "text" })
```

Search:

```
db.posts.find({ $text: { $search: "mongodb tutorial" } })
```

Useful for:

- ❑ Blogs
- ❑ Search features

Features:

1. Tokenization
   Sentence → words me tod diya jata hai
   "Mongo is powerful" → mongo, powerful

2. Stop Words ignore hote hain
   Words jaise "is", "the", "a" ignore ho jate hain

3. Stemming
   "running", "runs" → "run" treat hoga

4. Score (Relevance)

Most relevant result pehle aayega

```
db.articles.find(
  { $text: { $search: "MongoDB" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })
```

5. Multiple fields pe text index

```
db.articles.createIndex({
  title: "text",
  content: "text"
})
```

Dono fields me search hoga

Note: Text Score Factors:

- ❑ Term Frequency (TF)Word kitni baar aaya hai
- ❑ Field Length (Normalization)
  - Chhota field (title) → zyada impact
  - Bada field (content) → score dilute
- ❑ Match Quality
  - Exact word match vs weak match
- ❑ Number of matched terms
  - Kitne search words match hue

Important Limitations

1. Ek collection me sirf 1 text index
   But multiple fields include kar sakte ho

2. Exact phrase ke liye quotes

```
$search: "\"MongoDB index\""
```

3. Regex jaisa flexible nahi
   Ye full-text search hai, pattern matching nahi

More About Indexing

unique: true

Ensures duplicate values allow nahi honge

```
db.users.createIndex({ email: 1 }, { unique: true })
```

Use case: Email, Username, Phone number

Important:

- ❑ Agar collection me already duplicate hai → index create fail ho jayega
- ❑ Null bhi ek baar hi allowed hota hai (unless sparse/partial use karo)

partialFilterExpression

Index sirf selected documents par banta hai (full collection pe nahi)

```
db.users.createIndex(
  { age: 1 },
  { partialFilterExpression: { age: { $gt: 18 } } }
)
```

Sirf age > 18 wale docs indexed honge

Use case: Active users only, Paid users only, Filtered optimization

Benefit:

- ❑ Smaller index
- ❑ Faster writes (kyunki sab docs index nahi ho rahe)

expireAfterSeconds (TTL Index)

Auto delete documents after some time ⏳

```
db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 }
)
```

Use case: OTP, Sessions, Logs

Notes:

- ❑ Background process run hota hai (instant delete nahi hota)
- ❑ Sirf date field pe kaam karta hai

4. Covered Query

- ❑ Jab query ka saara data index se hi mil jaye
- ❑ DB ko actual document fetch hi nahi karna padta

```
db.users.createIndex({ name: 1, age: 1 })

db.users.find(
  { name: "Manas" },
  { name: 1, age: 1, _id: 0 }
)
```

Benefit: Super fast , Disk read kam

Covered nahi hoga agar:

- ❑ extra field mang li
- ❑ _id include ho gaya (by default hota hai)

Winning Plan (Query Planner)

- MongoDB decide karta hai kaunsa index use kare
- In case of multiple index on same field, MongoDB tests multiple indexes once, picks the fastest (winning plan), and caches it for future similar queries to avoid re-evaluation.

Index Cardinality

Unique values kitne hain field me. High cardinality fields are best for indexing

Example:

- gender → low cardinality
- email → high cardinality

Sparse Index

- Sirf un docs pe index banega jisme field exist karta hai
- Missing fields ignore honge

```
db.users.createIndex({ phone: 1 }, { sparse: true })
```

Index Performance Optimization

Check Index Usage

```
db.users.find({ email: "abc@gmail.com" }).explain("executionStats")
```

Important fields:

COLLSCAN ❌ (bad)

IXSCAN ☑ (good)

Tips:

Always index frequently queried fields

Use compound index smartly

Avoid unnecessary indexes

When to Use Indexes

- Large data (1000+ docs)
- Frequently searched fields
- Sorting and filtering

Note:

Companies always use indexing for:

- Search
- Filtering
- Recommendations

When NOT to Use Indexes

- When collection is small
- When Fields with low variation (like gender)
- When queries are complex (multiple fields)
- When the collection is frequently updated (more write operations)

Note:

Indexing slows write process.
indexing slows down: insert, update, delete

"Indexing is a trade-off:
Read fast, Write slow."

Thank You